

```
#!/usr/bin/env python
# coding: utf-8
```

```
# In[1]:
```

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from scipy.stats import uniform, randint
```

```
# In[2]:
```

```
import pandas as pd
import numpy as np
# Load the dataset
file_path = "C:/Users/dejif/OneDrive - University of
Stirling/Documents/ITNPBD6 Machine
Learning/Assignment/wallacecommunications.csv"
df = pd.read_csv(file_path)
```

```
# In[3]:
```

```
df_copy = df.copy()
```

```
# In[4]:
```

```
df.head(10)
```

```
# In[5]:
```

```
df.shape
```

```
# In[6]:
```

```
df.dtypes
```

```
# In[7]:
```

```
df.isnull().sum()
```

```
# In[8]:
```

```
# Plot distributions...  
df.hist(figsize = (15,20))
```

```
# In[9]:
```

```
# this will list the non-numeric variables  
# so loop over all of them; count each unique value, and plot...  
for col in df.select_dtypes(include=object).columns:  
    print(col)  
    df[col].value_counts().plot(kind='bar')  
    plt.show()
```

```
# In[10]:
```

```
df_cleaned = df.copy()  
# Choose the row to keep  
df_cleaned = df_cleaned[(df_cleaned.country == "UK") &  
    (df_cleaned.last_contact_this_campaign_month != "j")]  
df_cleaned['country'].value_counts().plot(kind='bar')
```

```
# In[11]:
```

```
df_cleaned['last_contact_this_campaign_month'].value_counts().plot(kind='bar')
```

```
# In[12]:
```

```
# Rename married column as marital_status  
df_cleaned.rename(columns={"married": "marital_status"}, inplace=True)  
df_cleaned['marital_status'].value_counts().plot(kind='bar')
```

```
# In[13]:
```

```
df_cleaned['has_tv_package'] =  
df_cleaned['has_tv_package'].replace({'n':'no'})  
df_cleaned['has_tv_package'].value_counts().plot(kind='bar')
```

```
# In[14]:
```

```
df_cleaned['last_contact'] =  
df_cleaned['last_contact'].replace({'cell':'cellular'})  
df_cleaned['last_contact'].value_counts().plot(kind='bar')
```

```
# In[15]:
```

```
# Drop unnecessary column  
# List of columns to drop  
columns_to_drop = ["ID", "town",  
"last_contact_this_campaign_day", "conn_tr", "contacted_during_previous_cam  
paign", "country",  
  
"days_since_last_contact_previous_campaign", "current_balance"]  
df_cleaned = df_cleaned.drop(columns= columns_to_drop)
```

```
# In[16]:
```

```
df_cleaned.reset_index(drop=True, inplace=True)
```

```
# In[17]:
```

```
df_cleaned
```

```
# In[18]:
```

```
# Convert target variable to binary  
df_cleaned["new_contract_this_campaign"] =  
df_cleaned["new_contract_this_campaign"].map({"yes": 1, "no": 0})
```

```
# In[19]:
```

```
# Identify categorical variables  
categorical_cols = df_cleaned.select_dtypes(include=["object"]).columns
```

```
# In[20]:
```

```
# Define features and target variable  
X = df_cleaned.drop(columns=["new_contract_this_campaign"])  
y = df_cleaned["new_contract_this_campaign"]
```

```
# In[21]:
```

```
# Split dataset into train (60%), validation (20%), and test (20%)
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.40,
random_state=42, stratify=y)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp,
test_size=0.50, random_state=42, stratify=y_temp)
```

```
# In[22]:
```

```
X_train
```

```
# In[23]:
```

```
y_train
```

```
# In[24]:
```

```
get_ipython().system('pip install imbalanced-learn')
```

```
# In[25]:
```

```
# Apply Label Encoding to categorical features BEFORE SMOTE
label_encoders = {}
X_train_encoded = X_train.copy()
X_val_encoded = X_val.copy()

for col in categorical_cols:
    le = LabelEncoder()
    X_train_encoded[col] = le.fit_transform(X_train_encoded[col])
    X_val_encoded[col] = le.transform(X_val_encoded[col])
    label_encoders[col] = le
```

```
# In[26]:
```

```
# Apply SMOTE on the encoded data
smote = SMOTE(random_state=42)
X_train_encoded, y_train = smote.fit_resample(X_train_encoded, y_train)
```

```
# In[27]:
```

```
pip install --upgrade scikit-learn
```

```
# In[28]:
```

```

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.preprocessing import OneHotEncoder

# Apply One-Hot Encoding for Logistic Regression and Standard Scaling
numerical_cols = ["age",
                  "this_campaign",]

preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numerical_cols),
    ("cat", OneHotEncoder(handle_unknown='ignore'), categorical_cols)
], remainder="passthrough")

X_train_log = preprocessor.fit_transform(X_train_encoded)
X_val_log = preprocessor.transform(X_val_encoded)
X_test_log = preprocessor.transform(X_test)

# In[29]:

print(f"Number of categorical columns: {len(categorical_cols)}")
print("Categorical columns:", categorical_cols.tolist())

# In[30]:

# Apply Label Encoding for Decision Tree & Neural Network
X_train_tree_nn = X_train_encoded.copy()
X_val_tree_nn = X_val_encoded.copy()
X_test_tree_nn = X_test.copy()

for col in categorical_cols:
    X_test_tree_nn[col] =
label_encoders[col].transform(X_test_tree_nn[col])

# In[31]:

# Standardize numerical columns for Decision Tree & Neural Network
scaler = StandardScaler()
X_train_tree_nn[numerical_cols] =
scaler.fit_transform(X_train_tree_nn[numerical_cols])
X_val_tree_nn[numerical_cols] =
scaler.transform(X_val_tree_nn[numerical_cols])
X_test_tree_nn[numerical_cols] =
scaler.transform(X_test_tree_nn[numerical_cols])

# In[32]:

# Initialize models
log_reg = LogisticRegression(max_iter=500, random_state=42)

```

```

tree_model = DecisionTreeClassifier(max_depth=10, random_state=42,
class_weight="balanced")
nn_model = MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=500,
random_state=42)

```

```

# In[33]:

```

```

# Train models
log_reg.fit(X_train_log, y_train)
tree_model.fit(X_train_tree_nn, y_train)
nn_model.fit(X_train_tree_nn, y_train)

```

```

# In[34]:

```

```

# Use RandomizedSearchCV for hyperparameter tuning on training data
get_ipython().system('pip install --upgrade scikit-learn joblib')

```

```

log_reg_search = RandomizedSearchCV(LogisticRegression(max_iter=500,
random_state=42),
                                param_distributions={'C': uniform(0.01,
10)}),
                                n_iter=10, cv=3, scoring='f1',

```

```

random_state=42)
log_reg_search.fit(X_train_log, y_train)

```

```

tree_model_search =
RandomizedSearchCV(DecisionTreeClassifier(random_state=42,
class_weight="balanced"),
                    param_distributions={
                        'max_depth': randint(5, 20),
                        'min_samples_split': randint(2, 10),
                        'min_samples_leaf': randint(1, 5)
                    },
                    n_iter=10, cv=3, scoring='f1',

```

```

random_state=42)
tree_model_search.fit(X_train_tree_nn, y_train)

```

```

# Improved Neural Network Model

```

```

nn_param_grid = {
    'hidden_layer_sizes': [(64,), (128,), (64, 32), (128, 64), (128, 64,
32)],
    'activation': ['relu', 'tanh'],
    'alpha': uniform(0.0001, 0.01),
    'learning_rate_init': uniform(0.001, 0.01)
}

```

```

nn_model_search = RandomizedSearchCV(
    MLPClassifier(max_iter=1500, random_state=42, early_stopping=True,
validation_fraction=0.1),
    param_distributions=nn_param_grid,
    n_iter=20, # Increased to search more combinations
    cv=3,
    scoring='f1',
    random_state=42,

```

```

        n_jobs=-1,
        verbose=1
    )

nn_model_search.fit(X_train_tree_nn, y_train)

# In[35]:

# Extract best models
best_log_reg = log_reg_search.best_estimator_
best_tree_model = tree_model_search.best_estimator_
best_nn_model = nn_model_search.best_estimator_

# In[36]:

# Make predictions on validation set
y_val_pred_log = best_log_reg.predict(X_val_log)
y_val_pred_tree = best_tree_model.predict(X_val_tree_nn)
y_val_pred_nn = best_nn_model.predict(X_val_tree_nn)

# In[37]:

# Evaluate models on Validation Set
# Function to evaluate models (MOVE THIS TO THE BEGINNING)
def evaluate_model(y_true, y_pred):
    return {
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred, zero_division=0),
        "Recall": recall_score(y_true, y_pred, zero_division=0),
        "F1-score": f1_score(y_true, y_pred, zero_division=0),
        "Confusion Matrix": confusion_matrix(y_true, y_pred)
    }
log_reg_val_results = evaluate_model(y_val, y_val_pred_log)
tree_val_results = evaluate_model(y_val, y_val_pred_tree)
nn_val_results = evaluate_model(y_val, y_val_pred_nn)

# In[38]:

# Print Validation Results
print("Validation Results:")
print("Logistic Regression:", log_reg_val_results)
print("Decision Tree:", tree_val_results)
print("Neural Network:", nn_val_results)

# In[39]:

# Make predictions on test set
y_pred_log = best_log_reg.predict(X_test_log)

```

```
y_pred_tree = best_tree_model.predict(X_test_tree_nn)
y_pred_nn = best_nn_model.predict(X_test_tree_nn)
```

```
# In[40]:
```

```
# Get metrics and confusion matrices
log_reg_results = evaluate_model(y_test, y_pred_log)
tree_results = evaluate_model(y_test, y_pred_tree)
nn_results = evaluate_model(y_test, y_pred_nn)
```

```
# In[41]:
```

```
# Plot confusion matrices
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, (title, results) in zip(axes, [("Logistic Regression",
                                     log_reg_results),
                                     ("Decision Tree", tree_results),
                                     ("Neural Network", nn_results)]):
    sns.heatmap(results["Confusion Matrix"], annot=True, fmt="d",
                cmap="Blues", ax=ax)
    ax.set_title(title)
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")

plt.show()

# Print results
log_reg_results, tree_results, nn_results
```

```
# In[42]:
```

```
# Get feature names from your encoded + scaled data
# For decision tree, you used LabelEncoder + StandardScaler
# So we'll just reuse the original column names (simplified version)
features = X_train_tree_nn.columns
```

```
# Get feature importances
importances = best_tree_model.feature_importances_
```

```
# Create a DataFrame for easier plotting
feature_importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': importances
}).sort_values(by='Importance', ascending=False)
```

```
# Plot top 10 features
plt.figure(figsize=(10, 6))
plt.barh(feature_importance_df['Feature'][:10],
         feature_importance_df['Importance'][:10])
plt.gca().invert_yaxis()
plt.title('Top 10 Important Features (Decision Tree)')
```



```
plt.xlabel('Importance Score')  
plt.tight_layout()  
plt.show()
```